Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Kod kusurlarını (code smell) kod üzerinde bulma ve adlandırma
  - Bir kodda kod kusuru, geçtiği yer ya da yerlerde işaretlenir; her kusura bir numara verilir ve ayrıca kusurun orijinal (İngilizce) adı ile onu gideren refactoring eyleminin orijinal adı yazılır. Örneğin "1 numaralı yerde Long Method var, Extract Method uygulanır" ya da "Pull Up Method uygulanır".
  - Bazı kusurlar tanımı gereği birden çok yerde görünür: Shotgun Surgery ve Divergent Change, birkaç yerde birden değişiklik yapma zorunluluğuyla ilgilidir. Kodun birkaç yerindeki aynı kopya kod (Duplicated Code) tek bir kod kusurudur.
  - Buna karşılık üç ayrı metodun her biri uzunsa bunlar üç ayrı Long Method kusurudur.
  - Düzeltilmiş kodun tasarımı UML sınıf şeması kurallarına uygun çizilmelidir.
- İkinci ara sınav, Soru 1: havayolu sadakat kartı (ekrandan, 00:10:00; çözüm notu ekrandan, 00:05:10)
  - Bronz, Gümüş, Altın üyelik düzeyleri; mil başına puan; yıllık puan sınırına ulaşamayan yolcu bir alt düzeye düşer; her düzeyin altı hizmetten hangilerinden yararlandığı bir tabloyla verilir.
  - Çözüm notunda tam uygun (+): State, Strategy ("ilk State düşünülür ama geçişler çok sık olmadığı için bu da olur"), Prototype. Tam uygun değil (±): Chain of Responsibility, Builder, Abstract Factory.
  - Derste Prototype'ın ayrıntılı tartışmasından sonra hoca Prototype'ı "yarım uygun" saymaya karar verdiğini söyledi.
- State kalıbının bu soruda üstünlüğü
  - State'te varlık kendi durum geçişlerini kendisi yapar. Puanı ya da üyelik kurallarını etkileyen bir metot çağrıldığında geçiş denetimi nesnenin içinde yapılır.
  - Öbür türlü, veri tabanında periyodik çalışan bir görev (zamanlanmış görev) ya da harici bir sorgulama süreci kurup bütün kayıtları tek tek tarayarak üyelik yükseltme ve düşürmeyi dışarıdan yapmak gerekir.
- Chain of Responsibility tartışması
  - Bir öğrenci üyelik düzeylerinin puana göre zincirle belirlenebileceğini, puanın kaç uçuştan geldiğinin önemsiz olduğunu savundu.
  - Hoca: Chain of Responsibility'de istek normalde zincirde bir üst seviyeye iletilir; düzey düşürmek için zincirin öbür yönüne de bağlantı kurmak gerekir. Belki "dörtte üç uygun" denebilirdi, ama en uygun kalıp State'tir.
- Prototype tartışması
  - Uygulanışı: yolcu nesnesinde üyelik türü nesnesi bir üye alan olarak tutulur; ona bronz, gümüş ya da altın prototipinden kopyalanan bir nesne atanır.
  - Singleton olmaz: iki farklı bronz üyenin farklı durum bilgileri (ör. puanları) olacaktır.
  - Dezavantaj 1 (öğrenci, hoca onayladı): Prototype değişmeyecek şeyler için uygundur (ör. oyunda bir canavar türünün kopyası). Burada üyelik düzeyi değişir; düşürme, yükseltme ve kampanya gösterme için ek mantık gerekir.
  - Dezavantaj 2 (hoca): altı hizmet için altı `boolean` üye alan gerekir; bronz prototipte 2 `true` 4 `false`, gümüşte 4 `true` 2 `false` olur. Çok uygun değildir.
  - Prototype'ta kalıtım yoktur: üye alanların durumları değiştirilerek farklı prototip örnekleri oluşturulur ve bir kayıt alanına (registry, havuz) kaydedilir. Uçuş kartından Bronz, Gümüş, Altın alt sınıfları türetip buna Prototype demek yanlıştır.
- İkinci ara sınav, Soru 2: üniversite ders kataloğu bilgi sistemi (ekrandan, 00:10:00 ve

00:21:20)
  - Sistemde çok sayıda ders vardır ve derslerin birçok ortak özelliği bulunur; bellek tasarrufu vurgulanmıştır.
  - Çözüm notu: “Bellek tasarrufunun önemli olduğu vurgulandığı için tek doğru çözüm Flyweight.” (ekrandan, 00:22:30)
  - Flyweight’te içsel (intrinsic) bilgi havuzda paylaşılır, bir kez bulunur; dışsal (extrinsic) bilgi bir milyon nesne varsa bir milyon kez tutulur.
  - Prototype da kopyalamada kolaylık sağlar, ama bellek tasarrufu vurgulanmasaydı uygun olurdu; bu yüzden yarı doğru sayıldı.
  - Abstract Factory: kalıtım gruplaması, yani kırılım (kategorileme) sorunları var. Builder: problem alanına uygun ama bellek tasarrufu sağlamaz; Singleton ile desteklenirse bellek tasarrufu sağlar ama bireysel ders nesnelerinin çeşitliliği desteklenmez. Prototype: aynı durum.
- Flyweight ile kod kalitesi arasındaki ödünleşim
  - Bir öğrenci Flyweight uygulamasının zorunlu ders mi, ödevi var mı, projesi var mı gibi denetimlerle if-else’e kaydığını söyledi. Hoca bunun kusurlu kod olabileceğini kabul etti.
  - Bellek öncelikliyse kısa vadede işletim ve donanım masrafını düşürmek için bu karar verilir. Bakım maliyeti de düşünülmelidir: daha iyi bir tasarımla bakım 1 birime mal olurken bu tasarımla 1,5 birime mal oluyorsa sonraki sürümde Flyweight bırakılıp donanım masrafı göze alınabilir ve maliyet ürün fiyatına yansıtılabilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Kalıtım yalnız gerektiğinde kullanılmalıdır; gerekmediği yerde kullanılmaz. Prototype’ı alt sınıflarla çizmek bu ilkenin ihlalidir.
- Kod kusurlarının ve refactoring eylemlerinin adları orijinal adlarıyla, doğru yazılmalıdır.
- Flyweight’in ayırt edici özelliği durum bilgisinin içsel (paylaşılan) ve dışsal (nesneye özgü) olarak ikiye ayrılmasıdır; bellek tasarrufu vurgulanan bir problemde bunun alternatifi yoktur.
- Bir kalıp iş mantığına ve kalıp mantığına göre doğru olsa da uygulama alanına göre yanlış ya da eksik bir modelleme üretebilir (Abstract Factory’deki kategorileme örneği).

Kısa Tekrar Notları

- Sadakat kartı: State (en uygun), Strategy, Prototype (yarım uygun); Chain of Responsibility, Builder, Abstract Factory tam uygun değil; Singleton yanlış.
- State: varlık kendi durum geçişini yapar; harici zamanlanmış göreve gerek kalmaz.
- Chain of Responsibility: istek zincirde yukarı iletilir; düşürme için ters yönde bağlantı gerekir.
- Prototype: kalıtım yok; prototip örnekleri durumla farklılaşır ve registry’ye kaydedilir.
- Ders kataloğu: bellek tasarrufu → Flyweight; içsel bilgi havuzda tek, dışsal bilgi nesne başına.
- Flyweight kodu bozabilir; bakım maliyeti ile donanım maliyeti arasında ödünleşim vardır.
- Duplicated Code birden çok yerde görünse de tek kusurdur; üç uzun metot üç ayrı Long Method’dur.

Detaylı Açıklamalar

- Kod kusurlarını sayma ve adlandırma
  - Kod incelemesinde her kusur kodda işaretlenir ve numaralandırılır; ardından kusurun adı ve önerilen refactoring eylemi yazılır (ör. Long Method → Extract Method). Bazı kusurlar tek bir yere değil, birçok yere yayılır: Shotgun Surgery’de tek bir değişiklik birçok yere dokunmayı gerektirir. Kopya kodun her kopyası ayrı kusur sayılmaz; hepsi tek bir Duplicated Code kusurudur. Öte yandan farklı metotların her birindeki uzunluk ayrı ayrı Long Method kusurlarıdır.
  - Ders sonrası not: Hoca Shotgun Surgery ile Divergent Change’i birlikte “birkaç yerde birden değişiklik yapma zorunluluğu” diye andı. Fowler’ın *Refactoring* kitabındaki tanımlar büyük olasılıkla şöyledir: Shotgun Surgery’de tek bir değişiklik birçok sınıfta

küçük değişiklikler gerektirir; Divergent Change'de ise tek bir sınıf farklı nedenlerle farklı biçimlerde değişir.

- Soru 1'in yapısı (ekrandan, 00:10:00)
  - Bronz düzeyde 100 mil başına 1 puan, Gümüş'te 4 puan, Altın'da 15 puan kazanılır. Her yıl 50 puan toplamayan Altın üye Gümüş'e, 25 puan toplayamayan Gümüş üye Bronz'a düşer; Bronz'dan aşağı düşülmez.
  - Hizmet tablosu: bülten aboneliği ve araba kiralama indirimi her düzeyde var; öncelikli check-in ve özel bekleme salonu Gümüş ve Altın'da var; öncelikli bagaj alımı ve uçuşta ücretsiz ikram yalnız Altın'da var.
  - Sorunun özü, bir varlığın zamanla değişen durumu ve bu duruma bağlı davranışıdır. Bu yüzden State ilk akla gelen kalıptır; durum geçişleri çok sık olmadığı için Strategy de kabul edilir.
- Prototype'ın doğru ve yanlış kullanımı
  - Doğru kullanımda tek bir sınıfın farklı durumlu örnekleri hazırlanır (bronz prototip, gümüş prototip, altın prototip) ve bir havuza kaydedilir; yeni nesne gerektiğinde ilgili prototip kopyalanır. Yolcunun üyelik alanına bu kopyalardan biri atanır.
  - Bu soruda Prototype'ın zayıflığı iki yönlüdür: değişen üyelik düzeyi ve düzeye bağlı kampanyalar için ek mantık gerekir; ayrıca hizmetler altı `boolean` alanla tutulur ve prototipler yalnız bu alanların değerleriyle ayrışır.
- Soru 2 ve Abstract Factory'nin kırılım sorunu (ekrandan, 00:21:20)
  - Çözüm notundaki UML şemasında soyut `DersFactory` sınıfının üç alt sınıfı vardır: `ServisDersiFactory`, `SeçimlikDersFactory`, `ZorunluDersFactory`. `ZorunluDersFactory`, `ZorunluDersProjesi` ve `ZorunluDersSınavı` ürünlerini oluşturur. Ürün hiyerarşileri: soyut `Proje` ← `ZorunluDersProjesi`, `SeçimlikDersProjesi`; soyut `Sınav` ← `ZorunluDersSınavı`, `SeçimlikDersSınavı`.
  - Şemadaki iki not sorunu gösterir: `ServisDersiFactory` için "Bunun zorunlu veya seçimlik olma durumu?" ve ürün hiyerarşileri için "Bu kategorileme (ders dönemi: güz, bahar; ders türü: zorunlu, seçimlik vb.) kalıp mantığına göre daha doğru, ama uygulama alanına göre yanlış bir modelleme."
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.